

CSLF: Mitigating Catastrophic Forgetting via Cognitive-Semantic and Latent-Feature Collaboration

Anonymous ACL submission

Abstract

Mitigating catastrophic forgetting in continual learning is crucial. Existing methods have made breakthroughs by separating and fine-tuning parameters for different tasks. However, we observe that parameter separation is not truly achieved. The interference among categorical distributions across task-specific models leads to performance degradation as the number of tasks increases. Additionally, their task classification only relies on parameter-level learning, neglecting the model’s inherent prior knowledge at semantic level. Inspired by the dual-process theory in cognitive science, we propose a novel architecture: Cognitive-Semantic and Latent-Feature dual-stream synergistic architecture (CSLF), which focuses on achieving correct task classification and true parameter separation. CSLF performs task classification prior to routing samples to task-specific models, avoiding the interference among categorical distributions generated from different task-specific models. Through dynamic task category semantic extraction module and task latent feature abstraction module, CSLF can fully leverage the model’s task semantic space and feature mapping space simultaneously. Moreover, an adaptive gated fusion mechanism is integrated in CSLF to better fuse information from both spaces, further improving CSLF’s task classification capability. Experiments across various downstream tasks demonstrate the superiority of CSLF, establishing it as a promising paradigm for continual learning by merging semantic and latent features.

1 Introduction

Continual adaptation is essential for deployed NLP systems, but training on new tasks often overwrites old knowledge and causes catastrophic forgetting (Wang et al., 2024; Zhang et al., 2023; He et al., 2021). Existing solutions include regularization (e.g., EWC (Kirkpatrick et al., 2017)), re-

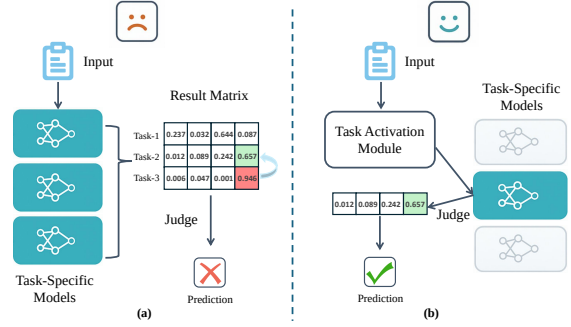


Figure 1: Right activate Task-Specific models to achieve true parameter isolation.

play (Jin et al., 2022), and, most effectively in recent LLM settings, parameter separation (Hu et al., 2021; Lester et al., 2021; Li and Liang, 2021; Guo et al., 2024; Singhal et al., 2022). Although separation reduces direct parameter conflict, current pipelines still suffer from non-trivial cross-task interference.

Firstly, samples from different tasks are often processed by multiple task-specific models in parallel, and the final prediction relies on combining outputs from all these models (Li et al., 2025a; Yang et al., 2024; Zhao et al., 2024). This design leads to inevitable interference among the categorical distributions generated by different separated models, especially as the number of tasks increases (Wang et al., 2025). For instance, when the training process progresses to the third task, where each task introduces four novel subclasses, the final result matrix has a dimension of 3 rows and 4 columns. As illustrated in the Figure 1 (a), the ground truth label of a sample corresponds to the fourth class in the second task (indicated in green). Examining exclusively the output from the task-2 model yields the correct prediction (with 0.657 being the maximum value). However, from a global perspective, the output value of 0.946 generated by the task-3 model surpasses the optimal solution de-

rived from task-2, thereby introducing interference. The primary reason for this phenomenon is that different task-specific models perform their respective softmax operations individually. Conversely, the human hippocampus first releases neurotransmitters to activate the correct neuronal regions, and only then proceeds to perform complex tasks based solely on the already activated neural circuits (Kinman et al., 2025). Therefore, we first select and activate the right task-specific model and then route samples to the chosen model, as illustrated in the Figure 1 (b).

In addition, current methods primarily perform task classification at the parameter level, overlooking the model’s inherent prior knowledge at the semantic level (Shen et al., 2023). In contrast, human cognition adheres to a dual-process principle: the intuitive system and the analytical system. The intuitive system relies on rapid neural electrical signal transmission, characterized by quick responses but low interpretability (Evans, 2008; De Martino et al., 2006). The analytical system establishes connections between new and existing knowledge through analogical learning at the semantic level. For example, when learning the concept of an “IP address”, it naturally draws an analogy with a “postal code”, thereby achieving better learning outcomes. This collaborative mechanism of dual systems ensures both rapid responsiveness and systematic knowledge construction. CSLF architecture draws inspiration from this principle. It dynamically harmonizes semantic knowledge and feature learning, outperforming conventional methods that rely solely on feature space representations. Therefore, our contributions are as follows:

- We propose a novel framework called the Cognitive-Semantic and Latent-Feature Dual-Stream Synergistic Architecture, which focuses on activating the correct task-specific models, enabling phased decision-making to achieve true parameter isolation and avoid interference between different tasks.
- Building upon the dual-process cognitive theory, we design key components, including a dynamic task category semantic extraction module and a task latent feature abstraction module. Our framework is the first to synergistically combine semantic knowledge with feature learning in the field of continual learning. It uniquely incorporates prior knowledge

from large language model pretraining into continual learning paradigms.

- We design an adaptive gated fusion mechanism to fully and efficiently fuse information from the two spaces, facilitating accurate and dynamic activation of task-specific models. Extensive experiments on multiple downstream tasks and comprehensive analyses demonstrate the superior performance of CSLF.

2 Related Work

2.1 Continual Learning

Traditional methods training large language models (LLMs) on static datasets struggle with dynamic real-world information (Brown et al., 2020; Tonmoy et al., 2024). Continual learning (Huang et al., 2021) enables continuous adaptation (Zhou et al., 2024), but faces the core challenge of catastrophic forgetting when integrating new knowledge. Existing solutions mainly fall into three categories. Regularization methods (Chen et al., 2020; Liu et al., 2023) penalize updates to previously critical weights, though inaccurate weight estimation can hinder new learning. Replay methods reinforce prior knowledge via historical data (Liu et al., 2021) or generated samples (Qin and Joty, 2021), which introduces privacy and storage overhead issues (Huang et al., 2024). In contrast, architecture-based methods (Zheng et al., 2024) leverage parameter-efficient fine-tuning to add new components or separate parameters for different tasks, achieving notable success in continual learning.

2.2 Methods Based on Parameter Separation

Parameter separation (Wang et al., 2023; Wistuba et al., 2023) is a crucial architecture-based approach in continual learning that effectively minimizes task interference. Furthermore, for massive models like LLaMA-65B (Touvron et al., 2023) and GLM-130B (Zeng et al., 2022), it avoids the prohibitive cost of full-parameter fine-tuning. We briefly overview two prominent separation methods. First, Adapters (Houlsby et al., 2019) insert lightweight bottleneck feed-forward networks between layers for efficient task adaptation, with variants like LAFT-URIEL (Badola et al., 2022) and TSS (Ke et al., 2023) enhancing them via feature guidance and soft-masking. Second, LoRA (Hu

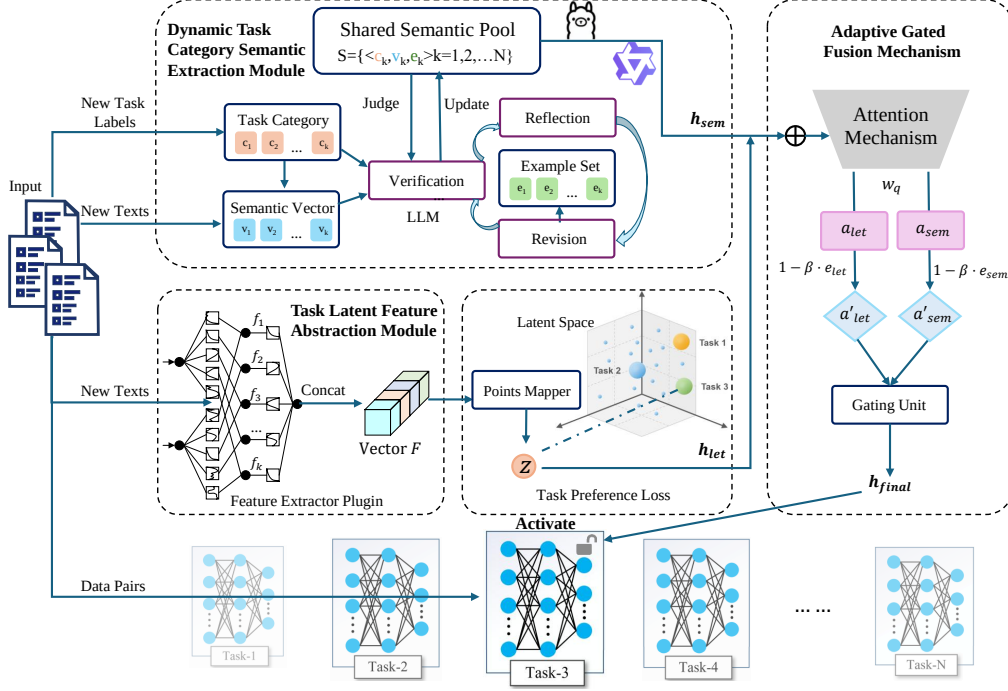


Figure 2: Complete Architecture of CSLF Framework showing the interaction between semantic extraction, latent feature processing, and adaptive fusion mechanisms.

et al., 2021) integrates trainable low-rank matrices into pre-trained layers for parameter-efficient updates. Frameworks like CoLoR (Wistuba et al., 2023) adopt LoRA for continual learning, while subsequent works (Li et al., 2025b; Liang and Li, 2024) further incorporate enhancements like data replay.

3 Methodology

Given a series of tasks $T_1, \dots, T_N$, where the total number of tasks is N , each task T_i introduces G new classes $Y_i = \{Y_i^1, \dots, Y_i^g, \dots, Y_i^G\}$. Each new class contains h labeled examples, represented as data pairs $(x_{i,g}^j, y_{i,g}^j, j = 1, \dots, h)$. The learner’s objective is to learn or optimize the current parameters Θ such that the average categorical loss across all tasks, $\frac{1}{N} \sum_{i \in \{1, \dots, N\}} \mathcal{L}(T_i; \Theta)$, is minimized. The challenge of continual learning lies in the fact that these tasks cannot be learned all at once, but must be acquired one by one.

The framework of the proposed CSLF architecture is illustrated in Figure 2. The architecture consists of three main modules: Dynamic Task Category Semantic Extraction Module, Task Latent Feature Abstraction Module, and adaptive Gated Fusion Mechanism to integrate both signals for task routing (Vaswani et al., 2017; Bahdanau et al., 2014).

Our framework handles multiple tasks where each task requires learning new class categories. At the start of task T_t , we inject its new class labels into the semantic stream and attach a unique task ID t to the corresponding task-specific adapter. The semantic stream produces a task-activation vector from category semantics and a prompted backbone prediction, while the latent stream produces another activation vector from intermediate hidden states. The two vectors have the same dimensionality as the number of observed tasks and are fused before selecting the activated task-specific model. Importantly, the semantic stream is used as a non-differentiable routing signal, whereas the latent stream is trained end-to-end with the active task adapter and the points mapper. This division avoids backpropagating through discrete prompting decisions while preserving a fully trainable latent routing path. Detailed prompts, tensor shapes, and optimization behavior are provided in Appendix H.

3.1 Dynamic Task Category Semantic Extraction Module

This module maintains a shared semantic pool $\mathcal{S} = \{\langle \mathbf{c}_k, \mathbf{v}_k, \mathbf{e}_k \rangle \mid k = 1, 2, \dots, N\}$, where $\mathbf{c}_k$ denotes the label texts associated with task k , $\mathbf{v}_k \in \mathbb{R}^{384}$ is the MiniLM embedding of the task description, and $\mathbf{e}_k \subseteq \mathcal{D}_k$ is a small exemplar

buffer used to refine the task description. Each entry points to a unique task ID rather than to a fine-grained class ID, because the goal of this module is routing across task-specific adapters.

3.1.1 Dynamic Category Semantic Extraction.

When a new task arrives, its categories $\mathbf{c}_{\text{new}} = \{c_{K+1}, \dots, c_{K+g}\}$ are dynamically injected into the semantic pool. The initial semantic vector for each new category is computed as

$$\mathbf{v}_{K+i} = \text{Embedding}(c_{K+i}), \quad i = 1, \dots, g \quad (1)$$

where Embedding is achieved through a light embedding model MiniLM-L12-v2.

This enables the model to maintain global category awareness. At the semantic level, the one-to-one correspondence between the model ID and the semantics of the categorical labels is established.

3.1.2 Similarity Calculation and Iterative Verification.

For a training batch $\mathcal{B} = \{x_1, \dots, x_N\}$, we compute the similarity between each sample and the semantic vectors in the pool. This is done by calculating the cosine similarity (Lahitani et al., 2016):

$$\text{Sim}(x_i, \mathbf{v}_k) = \frac{\text{Embedding}(x_i) \cdot \mathbf{v}_k}{\|\text{Embedding}(x_i)\|_2 \|\mathbf{v}_k\|_2} \quad (2)$$

Let $\mathbf{s}_i = [\text{Sim}(x_i, \mathbf{v}_1), \dots, \text{Sim}(x_i, \mathbf{v}_{|\mathcal{S}|})] \in \mathbb{R}^{|\mathcal{S}|}$ denote the similarity vector. We then build a prompted routing query by concatenating: (1) the input text x_i , (2) the top- m nearest task descriptions ranked by $\mathbf{s}_i$, and (3) at most b stored exemplars per candidate task. The backbone LLM is asked to output one task ID from the current candidate set. Concretely, $\text{Backbone}(x_i, \text{Sim}(x_i, \mathbf{v}_k))$ denotes this prompted forward pass and returns a task-score vector $\mathbf{h}_i \in \mathbb{R}^{|\mathcal{T}_{\leq t}|}$ over the observed tasks:

$$\mathbf{h}_i = \text{Backbone}(x_i, \text{Sim}(x_i, \mathbf{v}_k)) \quad (3)$$

The score vector is obtained from the normalized log-probabilities of the candidate task-ID strings. This semantic routing step uses the generative backbone only in forward mode, is not differentiated, and does not receive gradient updates from the routing loss. However, for entirely novel categories or fine-grained subcategories not encountered during pretraining, its performance tends to degrade. To address this, we employ an iterative verification-reflection-revision mechanism.

The system evaluates the activation accuracy as follows:

$$j^* = \arg \max_j \mathbf{h}_i[j], \quad r_i = \begin{cases} 1, & \text{if } j^* = k, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

The average of all samples in this batch is then calculated.

$$\bar{r} = \frac{1}{|\mathcal{B}|} \sum_{x_i \in \mathcal{B}} r_i \quad (5)$$

3.1.3 Reflection and Revision.

If $\bar{r}$ falls below the threshold, we revise the semantic pool with a deterministic reflection procedure. Specifically, we collect the misrouted samples in the current batch, group them by predicted/true task pair, retain the top- b highest-confidence errors per pair, and append them to the exemplar buffers $\mathbf{e}_k$. We then rebuild the task description by prompting the backbone to summarize: (1) the shared topic within each task, (2) the boundary to the most frequently confused neighboring task, and (3) short lexical cues extracted from the retained exemplars. The complete prompt template and update rules are reported in Appendix H. This loop improves semantic routing without changing the backbone parameters.

Our approach ensures fine-grained category representation accuracy through continuous evolution of the semantic pool

$$\mathcal{S}' = \mathcal{S} \cup \{\langle \mathbf{c}_{K+i}, \mathbf{v}_{K+i}, \mathbf{e}_{K+i} \rangle\} \quad (6)$$

In the inference stage, the backbone obtains the semantic-space activation vector $\mathbf{h}_{sem} \in \mathbb{R}^{|\mathcal{T}_{\leq t}|}$ based on the updated semantic pool $\mathcal{S}'$.

$$\mathbf{h}_{sem} = \text{Backbone}(\mathcal{S}', x_i) \quad (7)$$

For any input sentence x_i , the module outputs a semantic-space activation vector $\mathbf{h}_{sem}$. For a detailed overview of the training process, please refer to Algorithm 1.

3.2 Task Latent Feature Abstraction Module

Semantic routing can be unreliable when prior knowledge is weak. We therefore add a latent-feature stream for complementary task discrimination.

Specifically, the input text is fed into the active backbone together with the current task adapter. We extract hidden states from four evenly spaced intermediate layers (layers 6, 12, 18, and 24 for

Algorithm 1 Dynamic Task Category Semantic Extraction Module

Require: Existing semantic pool $\mathcal{S}$, new task categories $\mathbf{c}_{\text{new}} = \{c_{K+1}, \dots, c_{K+g}\}$, training batch $\mathcal{B}$

Ensure: Updated semantic pool $\mathcal{S}'$, semantic activation vector $\mathbf{h}_{\text{sem}}$

```

1: for each  $c_{K+i} \in \mathbf{c}_{\text{new}}$  do
2:   Compute semantic vector:  $\mathbf{v}_{K+i}$  via equation (1)
3:   Initialize representative samples:  $\mathbf{e}_{K+i} = \emptyset$ 
4: end for
5: for each  $x_i \in \mathcal{B}$  do
6:   for each  $\mathbf{v}_k \in \mathcal{S}'$  do
7:     Compute similarity:  $\text{Sim}(x_i, \mathbf{v}_k)$  via equation (2)
8:   end for
9:   Get initial activation:  $\mathbf{h}_i$  via equation (3)
10: end for
11: Evaluate activation accuracy for batch  $\mathcal{B}$ 
12: for each  $x_i \in \mathcal{B}$  do
13:   compute single sample accuracy via equation (4)
14: end for
15: Compute average accuracy via equation (5)
16: while  $\bar{r} < \tau$  do
17:   Perform reflection and revision:
18:   Record misclassified samples and update  $\mathbf{e}_{K+i}$ 
19:   Update semantic pool  $\mathcal{S}'$  via equation (6)
20:   Recompute activation and accuracy  $\bar{r}$ 
21: end while
22:  $\mathbf{h}_{\text{sem}} = \text{Backbone}(\mathcal{S}', x_i)$ 
23: return Updated semantic pool  $\mathcal{S}'$ , semantic activation vector  $\mathbf{h}_{\text{sem}}$ 

```

24-layer backbones; for backbones with a different depth, we use the closest quartile layers). From each selected layer we apply mean pooling over the non-padding tokens to obtain $\mathbf{f}_i \in \mathbb{R}^{d_{\text{hid}}}$. For token-level tasks, mean pooling is restricted to the marked entity span. The composite feature representation is then built by concatenation:

$$\mathbf{F} = \text{Concat}(\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_k) \in \mathbb{R}^{\sum_{i=1}^k d_i} \quad (8)$$

where $k = 4$, $d_i = d_{\text{hid}}$, and $\text{Concat}(\cdot)$ denotes feature concatenation. The resulting vector $\mathbf{F} \in \mathbb{R}^{4d_{\text{hid}}}$ retains both shallow and deep signals.

These fused features $\mathbf{F}$ are then input into the points mapper. Let the mapping function of this points mapper be $\mathcal{M} : \mathbb{R}^D \rightarrow \mathbb{R}^H$ ($D = 4d_{\text{hid}}$ and

$H = 256$). In practice, $\mathcal{M}$ is a two-layer MLP with dimensions $D \rightarrow 512 \rightarrow 256$, GELU activation, and LayerNorm after each linear projection. The mapped feature is:

$$\mathbf{z} = \mathcal{M}(\mathbf{F}) \in \mathbb{R}^H \quad (9)$$

In this high-dimensional vector space $\mathbb{R}^H$, each task T is assigned a unique coordinate position $\mathbf{p}_T \in \mathbb{R}^H$. A new point is created whenever a new task arrives. All task points are learnable, initialized from $\mathcal{N}(0, 0.02^2)$, and jointly optimized with $\mathcal{M}$ and the active task adapter.

To enhance task discrimination, we introduce a task preference loss $\mathcal{L}_{\text{task}}$ that uses the known training task ID as supervision. It optimizes the feature representation $\mathbf{z}$ of the current task T to be close to the coordinate point $\mathbf{p}_T$ while being far from the points $\mathbf{p}_{T'}$ of other tasks $T' \in \mathcal{T} \setminus \{T\}$:

$$\begin{aligned} \mathcal{L}_{\text{task}} = & \underbrace{\|\mathbf{z} - \mathbf{p}_T\|_2^2}_{\text{attraction}} \\ & + \lambda \cdot \underbrace{\sum_{T' \in \mathcal{T} \setminus \{T\}} \max(0, \gamma - \|\mathbf{z} - \mathbf{p}_{T'}\|_2^2)}_{\text{repulsion}} \end{aligned} \quad (10)$$

where $\|\cdot\|_2^2$ is the squared Euclidean distance, $\lambda > 0$ is the weight of the repulsion term, and $\gamma > 0$ is the minimum margin threshold. By minimizing this loss:

$$\min_{\theta, \{\mathbf{p}_T\}} \mathcal{L}_{\text{task}} \quad (11)$$

where θ are the learnable parameters of the points mapper, we obtain supervised task-ID separation in latent space. We emphasize this point because the loss requires the current sample's task membership during training; therefore it should be understood as a supervised auxiliary routing loss rather than an unsupervised separation objective.

The distances from the mapped feature to all task points are calculated as follows:

$$\mathbf{d}_T = \|\mathbf{z} - \mathbf{p}_T\|_2^2 \quad (12)$$

The latent feature space activation vector $\mathbf{h}_{\text{lat}} \in \mathbb{R}^{|\mathcal{T}_{\leq t}|}$ is then derived by inverse min-max normalization on $\mathbf{d}_T$, so smaller distances correspond to larger routing scores.

3.3 Adaptive Gated Fusion Mechanism

Semantic and latent streams provide complementary activations with input-dependent reliability.

We use attention with error-aware correction to reweight them online.

First, $\mathbf{h}_{sem}$ and $\mathbf{h}_{lat}$ are concatenated into a fused feature $\mathbf{h}_{concat} = [\mathbf{h}_{sem}; \mathbf{h}_{lat}]$, which serves as the input to the scorer. Then, compute the attention weights by introducing a learnable attention query vector $\mathbf{w}_q$ and calculating the attention for each vector via inner product:

$$[\alpha_{sem}, \alpha_{lat}] = \text{softmax} \left(\left[\mathbf{w}_q^\top \cdot \mathbf{h}_{sem}, \mathbf{w}_q^\top \cdot \mathbf{h}_{lat} \right] \right) \quad (13)$$

where α_{sem} and α_{lat} are the initial weights for the semantic and latent feature spaces, respectively, satisfying $\alpha_{sem} + \alpha_{lat} = 1$.

To avoid mismatches between initial attention weights and their real-world activation performance, we introduce an **error feedback correction term**. During training, the activation error rates of the two vectors, denoted as e_{sem} and e_{lat} , are tracked. The corrected weights are computed as:

$$\begin{aligned} \alpha'_{sem} &= \alpha_{sem} \cdot (1 - \beta \cdot e_{sem}) \\ \alpha'_{lat} &= \alpha_{lat} \cdot (1 - \beta \cdot e_{lat}) \end{aligned} \quad (14)$$

where $\beta \in (0, 1)$ is the error penalty coefficient, ensuring that vectors with higher error rates have their weights significantly reduced (e.g., if the semantic space activation has more errors, e_{sem} increases and α'_{sem} is automatically attenuated). Based on the corrected weights α'_{sem} and α'_{lat} , the final activation vector is generated via a gating unit:

$$\mathbf{h}_{final} = \alpha'_{sem} \cdot \mathbf{h}_{sem} + \alpha'_{lat} \cdot \mathbf{h}_{lat} \quad (15)$$

The core function of the gating unit is to achieve “dynamic selection” rather than simple weighting: when the error rate of one space remains high (e.g., $e_{sem} \rightarrow 1$), its weight $\alpha'_{sem} \rightarrow 0$, and the final output approximates the latent feature space vector; conversely, if the latent space is less reliable, the output emphasizes the semantic space vector. This realizes an “adaptive winner-takes-most” adjustment.

$\mathbf{h}_{final}$ is then used for the final classification task, where the task-specific model ID is determined by the maximum value in the vector:

$$\text{Model ID} = \arg \max_i \mathbf{h}_{final}[i] \quad (16)$$

During the inference phase, the task-specific model ID for the current sample is first determined using

$\mathbf{h}_{final}$. The sample is then routed to the corresponding task-specific model for prediction, yielding the final classification result.

4 Experiments

4.1 Experimental Settings

We conduct extensive experiments to evaluate the performance of the CSLF framework across various downstream tasks.

4.1.1 Datasets.

To comprehensively evaluate the effectiveness and generalizability of CSLF, we conduct experiments on four widely-used benchmark datasets spanning diverse downstream tasks: intent recognition, relation extraction, named entity recognition, and text classification. Specifically, we use Banking77 (Casanueva et al., 2020) for intent detection in the banking domain, FewRel (Han et al., 2018) for relation extraction (80 classes, 8 tasks, 10 classes per increment), Few-NERD (Ding et al., 2021) for fine-grained named entity recognition (66 classes, 11 tasks, 6 classes per increment), and CLINC150 (Ethayarajh, 2019) for multi-domain text classification (150 classes, 15 tasks, 10 classes per increment). This broad selection covers both sentence-level and token-level continual learning scenarios, ensuring a thorough and robust assessment of CSLF across a wide range of real-world applications.

4.1.2 Baselines.

We compare CSLF with several strong baselines, including classic regularization-based and replay-based methods, as well as the latest parameter separation approaches. Parameter separation is mainly implemented via Adapters and LoRA techniques, establishing a more comprehensive comparison framework.

SEQ (Sequential Fine-tuning) serves as the lower bound for continual learning performance; EWC (Kirkpatrick et al., 2017), a classic regularization-based method, mitigates forgetting by selectively slowing down the learning of weights important for previous tasks; DER++ (Buzzege et al., 2020), a representative replay-based method, maintains performance by replaying stored samples from previous tasks; DMEA (Qin et al., 2023) implements task-specific parameter separation using the Adapter technique; InfLoRA (Liang and Li, 2024) achieves parameter separation via LoRA,

Model	Banking77		FewRel		Few-NERD		CLINC150	
	$Score_{N-1}$	$Score_N$	$Score_{N-1}$	$Score_N$	$Score_{N-1}$	$Score_N$	$Score_{N-1}$	$Score_N$
Llama-3-8B								
SEQ	15.98	14.28	13.23	9.64	10.86	9.25	7.14	6.67
EWC	40.41	38.40	32.25	29.18	37.84	33.02	48.03	45.39
DER++	51.16	49.82	49.21	47.90	48.63	47.21	49.67	48.33
DMEA	60.20	58.91	67.75	65.41	64.12	<u>62.97</u>	<u>70.02</u>	<u>68.88</u>
InfLoRA	<u>67.81</u>	<u>63.32</u>	65.04	61.93	<u>65.32</u>	60.12	68.32	64.20
I-LoRA	65.44	61.03	71.34	68.82	62.89	59.73	61.31	60.16
CSLF(Ours)	75.12	74.01	<u>70.44</u>	69.38	72.67	69.55	78.83	75.74
Qwen-3-14B								
SEQ	17.45	15.12	15.23	11.21	10.31	8.97	9.02	8.55
EWC	41.67	39.88	36.41	32.85	39.12	34.28	49.23	47.01
DER++	53.12	51.34	51.09	49.82	50.56	49.18	52.13	50.97
DMEA	62.89	60.41	63.02	61.77	64.01	62.62	<u>75.01</u>	<u>72.88</u>
InfLoRA	68.81	67.12	66.23	64.91	65.12	63.73	66.12	64.94
I-LoRA	<u>70.03</u>	<u>68.87</u>	<u>71.01</u>	<u>69.62</u>	<u>71.09</u>	<u>69.41</u>	71.21	69.98
CSLF(Ours)	78.21	75.18	72.12	70.93	73.23	71.98	82.01	80.77

Table 1: Performance comparison of different methods on four continual learning datasets. The $Score_{N-1}$ represents the average accuracy when training up to the second-to-last task, and the $Score_N$ represents the average accuracy when training up to the last task. The best results are in **bold**, and the second-best results are underlined.

originally proposed for vision tasks, and we re-implement it for NLP tasks; I-LoRA (Li et al., 2025b) combines data replay and LoRA-based parameter separation to improve continual learning.

4.1.3 Implementation Details.

CSLF is trained on a single NVIDIA A100 GPU with 80GB memory. The training batch size is set from 4 to 16, and the learning rate is initialized at $1e-5$. The task-specific model employs LoRA for efficient parameter adaptation. The rank of LoRA is 4, the scaling factor is 8, and the dropout rate is 0.1. In the semantic stream, MiniLM-L12-v2 is used only for 384-dimensional similarity retrieval, and the backbone LLM is queried with a fixed routing prompt to produce task-ID scores without gradient updates. In the latent stream, the backbone forward pass is differentiated through the active LoRA adapter, while the frozen backbone weights remain unchanged. We extract hidden states from four intermediate layers, mean-pool them, concatenate them, and map them to a 256-dimensional routing space with a two-layer MLP. Additional prompt templates and optimization details are given in Appendix H.

4.1.4 Backbone.

We use the Llama-3-8b model (Grattafiori et al., 2024) and Qwen3-14B model (Bai et al., 2023) as the backbone for CSLF. These models are chosen for their strong performance in various NLP tasks and their ability to handle large-scale data efficiently. The models are pre-trained on extensive corpora, providing a solid foundation for continual learning.

4.1.5 Evaluation Metrics.

In continual learning, several key metrics are used to evaluate model performance as new tasks are learned sequentially.

Average Accuracy (AA) measures the mean accuracy across all tasks after the model has completed learning the entire sequence, providing an overall assessment of knowledge retention and performance on all tasks:

$$AA = \frac{1}{N} \sum_{i=1}^N Acc(i, N) \quad (17)$$

where N is the total number of tasks, and $Acc(i, N)$ denotes the accuracy on task i after learning all N tasks.

Backward Transfer (BWT) quantifies the influence of learning new tasks on the performance of previously learned tasks. Negative BWT indicates forgetting, while positive BWT suggests that learning new tasks helps retain or even improve old knowledge:

$$BWT = \frac{1}{N-1} \sum_{i=1}^{N-1} [Acc(i, N) - Acc(i, i)] \quad (18)$$

Forward Transfer (FWT) evaluates how prior knowledge affects the learning of new tasks. Positive FWT means that previous learning facilitates adaptation to new tasks, while negative FWT indicates interference:

$$FWT = \frac{1}{N-1} \sum_{j=2}^N [Acc(j, j-1) - Acc_{scratch}(j)] \quad (19)$$

4.2 Main Results

The experimental results in Table 1 highlight CSLF’s superior performance across diverse downstream tasks, with particularly strong results on CLINC150, achieving 78.83%/75.74%, significantly outperforming the best baseline’s 70.02%/68.88%. On Banking77, CSLF achieves 75.12%/74.01%, surpassing the second-best method by a margin of over 7%. Similarly, for FewRel and Few-NERD, CSLF demonstrates competitive performance, achieving 70.44%/69.38% and 72.67%/69.55%, respectively, consistently outperforming other methods. These results indicate that CSLF effectively mitigates catastrophic forgetting while maintaining strong adaptability to new tasks.

Method	BWT	FWT
SEQ	-43.28	10.13
EWC	-6.34	12.32
DER++	-8.38	13.46
DMEA	-4.24	13.57
InfLoRA	-3.25	15.13
I-LoRA	-3.47	16.25
CSLF (Ours)	-2.74	16.82

Table 2: BWT (Backward Transfer) and FWT (Forward Transfer) results of CSLF and baseline methods on Llama-3-8B model.

The improvements are consistent across both Llama-3-8B and Qwen-3-14B backbones, show-

ing the robustness and generalizability of our dual-stream architecture. Notably, the dynamic task category semantic extraction module enables CSLF to leverage prior knowledge for better task classification, while the task latent feature abstraction module provides complementary discriminative features. The adaptive gated fusion mechanism further enhances performance by dynamically balancing the contributions of semantic and latent spaces. These results validate CSLF’s ability to achieve true parameter isolation and avoid interference among tasks, making it a promising paradigm for continual learning across various NLP tasks.

Table 2 presents BWT and FWT results for CSLF and baseline methods on the Llama-3-8B model. As a negative baseline, SEQ suffers from severe catastrophic forgetting, as evidenced by its large negative BWT and FWT, highlighting the limitations of naive sequential learning. Classic strategies such as EWC and DER++ alleviate forgetting compared to SEQ. Overall, these classic approaches vary in their ability to balance old and new tasks, but all lack targeted optimization. Parameter separation methods (such as DMEA, InfLoRA, and I-LoRA) outperform classic strategies by significantly reducing forgetting and improving new task gains, demonstrating the inherent advantage of parameter isolation in balancing knowledge retention and acquisition. Our proposed CSLF achieves the best performance, minimizing forgetting (BWT closest to zero) and maximizing new task gains (highest FWT), thus perfectly balancing the needs of both old and new tasks and validating its effectiveness for continual learning scenarios.

4.3 Ablation Study and Analysis

To evaluate the components of CSLF, we conducted ablation studies on the Banking77 dataset (Table 3). Results show that the semantic component effectively leverages inherent knowledge for task classification, while the latent feature component provides complementary information. Specifically, the semantic module alone yields significant improvements over baselines, highlighting the value of pre-trained knowledge, whereas the latent module excels in capturing abstract boundaries. Combining both components significantly improves performance, suggesting they capture orthogonal task characteristics. Furthermore, adding the attention mechanism enables dynamic weighting for better classification. The error-aware gating mechanism enhances adaptability by suppressing unreliable sig-

nals and selecting the most reliable stream. The full CSLF framework achieves the best performance, demonstrating synergistic benefits.

Components	AA	BWT	ΔAA
<i>Baselines</i>			
SEQ	15.98	-44.09	-
I-LoRA	65.44	-3.03	+49.46
<i>CSLF Components</i>			
Semantic (S)	58.23	-8.45	+42.25
Latent (L)	52.17	-12.34	+36.19
S + L (Simple)	67.89	-5.21	+51.91
+ Attention	70.45	-4.12	+54.47
+ Gating	72.78	-3.67	+56.80
CSLF (Full)	75.12	-2.74	+59.14

Table 3: Ablation study on Banking77 with Llama-3-8B. AA: Average Accuracy; BWT: Backward Transfer; ΔAA: improvement over SEQ baseline.

5 Discussion and Conclusions

We introduced CSLF, a dual-stream continual learning framework that performs semantic-aware task routing and latent-feature discrimination before activating task-specific adapters. Across four datasets and two LLM backbones, CSLF improves accuracy, reduces forgetting, and increases forward transfer over strong baselines.

References

Kartikeya Badola, Shachi Dave, and Partha Talukdar. 2022. Parameter-efficient finetuning for robust continual multilingual learning. *arXiv preprint arXiv:2209.06767*.

Bahdanau, Dzmitry, Cho, Kyunghyun, Bengio, and Yoshua. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, and 29 others. 2023. *Qwen technical report*. *Preprint*, arXiv:2309.16609.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient intent detection with dual sentence encoders. *arXiv preprint arXiv:2003.04807*.

Sanyuan Chen, Yutai Hou, Yiming Cui, Wanxiang Che, Ting Liu, and Xiangzhan Yu. 2020. Recall and learn: Fine-tuning deep pretrained language models with less forgetting. *arXiv preprint arXiv:2004.12651*.

Benedetto De Martino, Dharshan Kumaran, Ben Seymour, and Raymond J Dolan. 2006. Frames, biases, and rational decision-making in the human brain. *science*, 313(5787):684–687.

Ning Ding, Guangwei Xu, Yulin Chen, Xiaobin Wang, Xu Han, Pengjun Xie, Hai-Tao Zheng, and Zhiyuan Liu. 2021. Few-nerd: A few-shot named entity recognition dataset. *arXiv preprint arXiv:2105.07464*.

Kawin Ethayarajh. 2019. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings. *arXiv preprint arXiv:1909.00512*.

Jonathan St BT Evans. 2008. Dual-processing accounts of reasoning, judgment, and social cognition. *Annu. Rev. Psychol.*, 59(1):255–278.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 41 others. 2024. *The llama 3 herd of models*. *Preprint*, arXiv:2407.21783.

Jiafeng Guo, Changjiang Zhou, Ruqing Zhang, Jiangui Chen, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. 2024. *Corpusbrain++: A continual generative pre-training framework for knowledge-intensive language tasks*. *Preprint*, arXiv:2402.16767.

Xu Han, Hao Zhu, Pengfei Yu, Ziyun Wang, Yuan Yao, Zhiyuan Liu, and Maosong Sun. 2018. Fewrel: A large-scale supervised few-shot relation classification dataset with state-of-the-art evaluation. *arXiv preprint arXiv:1810.10147*.

Tianxing He, Jun Liu, Kyunghyun Cho, Myle Ott, Bing Liu, James Glass, and Fuchun Peng. 2021. *Analyzing the forgetting problem in the pretrain-finetuning of dialogue response models*. *Preprint*, arXiv:1910.07117.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019.

670	Parameter-efficient transfer learning for nlp. In <i>International conference on machine learning</i> , pages 2790–2799. PMLR.	725
671		726
672		727
673	Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan	728
674	Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and	729
675	Weizhu Chen. 2021. Lora: Low-rank adaptation of	730
676	large language models . <i>Preprint</i> , arXiv:2106.09685.	731
677	Jianheng Huang, Leyang Cui, Ante Wang, Chengyi	732
678	Yang, Xinting Liao, Linfeng Song, Junfeng Yao, and	733
679	Jinsong Su. 2024. Mitigating catastrophic forget-	
680	ting in large language models with self-synthesized	
681	rehearsal. <i>arXiv preprint arXiv:2403.01244</i> .	
682	Yufan Huang, Yanzhe Zhang, Jiaao Chen, Xuezhi Wang,	
683	and Diyi Yang. 2021. Continual learning for text clas-	
684	sification with information disentanglement based	
685	regularization. <i>arXiv preprint arXiv:2104.05489</i> .	
686	Xisen Jin, Dejiao Zhang, Henghui Zhu, Wei Xiao,	
687	Shang-Wen Li, Xiaokai Wei, Andrew Arnold, and	
688	Xiang Ren. 2022. Lifelong pretraining: Continu-	
689	ally adapting language models to emerging corpora .	
690	<i>Preprint</i> , arXiv:2110.08534.	
691	Zixuan Ke, Bing Liu, Wenhan Xiong, Asli Celikyilmaz,	
692	and Haoran Li. 2023. Sub-network discovery and	
693	soft-masking for continual learning of mixed tasks.	
694	<i>arXiv preprint arXiv:2310.09436</i> .	
695	Adrienne I Kinman, Derek N Merryweather, Sarah R Er-	
696	win, Regan E Campbell, Kaitlin E Sullivan, Larissa	
697	Kraus, Margarita Kapustina, Brianna N Bristow,	
698	Mingjia Y Zhang, Madeline W Elder, and 1 others.	
699	2025. Atypical hippocampal excitatory neurons ex-	
700	press and govern object memory. <i>Nature Communi-</i>	
701	<i>cations</i> , 16(1):1195.	
702	James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz,	
703	Joel Veness, Guillaume Desjardins, Andrei A. Rusu,	
704	Kieran Milan, John Quan, Tiago Ramalho, Ag-	
705	nieszka Grabska-Barwinska, Demis Hassabis, Clau-	
706	dia Clopath, Dharshan Kumaran, and Raia Hadsell.	
707	2017. Overcoming catastrophic forgetting in neural	
708	networks . <i>Proceedings of the National Academy of</i>	
709	<i>Sciences</i> , 114(13):3521–3526.	
710	Lahitani, Alfirna Rizqi, Permanasari, Adhistya Erna, Se-	
711	tiawan, and Noor Akhmad. 2016. Cosine similarity	
712	to determine similarity measure: Study case in online	
713	essay assessment. In <i>2016 4th International confer-</i>	
714	<i>ence on cyber and IT service management</i> , pages 1–6.	
715	IEEE.	
716	Brian Lester, Rami Al-Rfou, and Noah Constant. 2021.	
717	The power of scale for parameter-efficient prompt	
718	tuning. <i>arXiv preprint arXiv:2104.08691</i> .	
719	Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning:	
720	Optimizing continuous prompts for generation. <i>arXiv</i>	
721	<i>preprint arXiv:2101.00190</i> .	
722	Xinlong Li, Weijieying Ren, Wei Qin, Lei Wang, Tianx-	
723	iang Zhao, and Richang Hong. 2025a. Analyzing	
724	and reducing catastrophic forgetting in parameter	
	efficient tuning . In <i>ICASSP 2025 - 2025 IEEE Inter-</i>	725
	<i>national Conference on Acoustics, Speech and Signal</i>	726
	<i>Processing (ICASSP)</i> , pages 1–5.	727
	Xinlong Li, Weijieying Ren, Wei Qin, Lei Wang, Tianx-	728
	iang Zhao, and Richang Hong. 2025b. Analyzing	729
	and reducing catastrophic forgetting in parameter	730
	efficient tuning. In <i>ICASSP 2025-2025 IEEE Interna-</i>	731
	<i>tional Conference on Acoustics, Speech and Signal</i>	732
	<i>Processing (ICASSP)</i> , pages 1–5. IEEE.	733
	Yan-Shuo Liang and Wu-Jun Li. 2024. Inflora:	734
	Interference-free low-rank adaptation for continual	735
	learning. In <i>Proceedings of the IEEE/CVF Confer-</i>	736
	<i>ence on Computer Vision and Pattern Recognition</i> ,	737
	pages 23638–23647.	738
	Qingbin Liu, Pengfei Cao, Cao Liu, Jiansong Chen,	739
	Xunliang Cai, Fan Yang, Shizhu He, Kang Liu, and	740
	Jun Zhao. 2021. Domain-lifelong learning for dia-	741
	logue state tracking via knowledge preservation net-	742
	works. In <i>Proceedings of the 2021 conference on</i>	743
	<i>empirical methods in natural language processing</i> ,	744
	pages 2301–2311.	745
	Qingbin Liu, Yanchao Hao, Xiaolong Liu, Bo Li, Di-	746
	anbo Sui, Shizhu He, Kang Liu, Jun Zhao, Xi Chen,	747
	Ningyu Zhang, and 1 others. 2023. Class lifelong	748
	learning for intent detection via structure consolida-	749
	tion networks. In <i>Findings of the Association for</i>	750
	<i>Computational Linguistics: ACL 2023</i> , pages 293–	751
	306.	752
	Chengwei Qin, Chen Chen, and Shafiq Joty. 2023.	753
	Lifelong sequence generation with dynamic mod-	754
	ule expansion and adaptation. <i>arXiv preprint</i>	755
	<i>arXiv:2310.09886</i> .	756
	Chengwei Qin and Shafiq Joty. 2021. Lfpt5: A uni-	757
	fied framework for lifelong few-shot language learn-	758
	ing based on prompt tuning of t5. <i>arXiv preprint</i>	759
	<i>arXiv:2110.07298</i> .	760
	Yikang Shen, Zheyu Zhang, Tianyou Cao, Shawn Tan,	761
	Zhenfang Chen, and Chuang Gan. 2023. Module-	762
	former: Modularity emerges from mixture-of-experts .	763
	<i>Preprint</i> , arXiv:2306.04640.	764
	Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara	765
	Mahdavi, Jason Wei, Hyung Won Chung, Nathan	766
	Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen	767
	Pfohl, Perry Payne, Martin Seneviratne, Paul Gam-	768
	ble, Chris Kelly, Nathaneal Scharli, Aakanksha	769
	Chowdhery, Philip Mansfield, Blaise Aguerre y Ar-	770
	cas, Dale Webster, and 11 others. 2022. Large lan-	771
	guage models encode clinical knowledge . <i>Preprint</i> ,	772
	arXiv:2212.13138.	773
	S. M Towhidul Islam Tonmoy, S M Mehedi Zaman,	774
	Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha,	775
	and Amitava Das. 2024. A comprehensive survey of	776
	hallucination mitigation techniques in large language	777
	models . <i>Preprint</i> , arXiv:2401.01313.	778
	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier	779
	Martinet, Marie-Anne Lachaux, Timothée Lacroix,	780

- Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and 1 others. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *CoRR*, abs/1706.03762.
- Huiyi Wang, Haodong Lu, Lina Yao, and Dong Gong. 2025. [Self-expansion of pre-trained models with mixture of adapters for continual learning](#). *Preprint*, arXiv:2403.18886.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 2024. [A comprehensive survey of continual learning: Theory, method and application](#). *Preprint*, arXiv:2302.00487.
- Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuan-jing Huang. 2023. Orthogonal subspace learning for language model continual learning. *arXiv preprint arXiv:2310.14152*.
- Martin Wistuba, Prabhu Teja Sivaprasad, Lukas Balles, and Giovanni Zappella. 2023. Continual learning with low rank adaptation. *arXiv preprint arXiv:2311.17601*.
- Shu Yang, Muhammad Asif Ali, Cheng-Long Wang, Lijie Hu, and Di Wang. 2024. [Moral: Moe augmented lora for llms’ lifelong learning](#). *Preprint*, arXiv:2402.11260.
- Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, and 1 others. 2022. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414*.
- Zihan Zhang, Meng Fang, Ling Chen, Mohammad-Reza Namazi-Rad, and Jun Wang. 2023. [How do large language models capture the ever-changing world knowledge? a review of recent advances](#). *Preprint*, arXiv:2310.07343.
- Weixiang Zhao, Shilong Wang, Yulin Hu, Yanyan Zhao, Bing Qin, Xuanyu Zhang, Qing Yang, Dongliang Xu, and Wanxiang Che. 2024. [Sapt: A shared attention framework for parameter-efficient continual learning of large language models](#). *Preprint*, arXiv:2401.08295.
- Junhao Zheng, Qianli Ma, Zhen Liu, Binqun Wu, and Huawen Feng. 2024. Beyond anti-forgetting: Multimodal continual instruction tuning with positive forward transfer. *arXiv preprint arXiv:2401.09181*.
- Da-Wei Zhou, Hai-Long Sun, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan. 2024. Continual learning with pre-trained models: A survey. *arXiv preprint arXiv:2401.16386*.

A The Theoretical Foundation of Cognitive Neuroscience

The dual-process theory of cognitive neuroscience is a pivotal framework in psychology and cognitive neuroscience that explains the mechanisms underlying human thinking and decision-making. Its core proposition is that human cognitive processes are executed through the synergistic interaction of two fundamentally distinct systems (or processing modes). The theory posits that human cognitive activities are governed by two systems: “System 1” (the intuitive system) and “System 2” (the analytical system), which differ significantly in terms of processing styles, functions, and neural substrates. System 1 (intuitive system) is characterized by rapid, automatic, unconscious processing that requires no deliberate effort, relying on experience and intuition for judgment—such as facial recognition or understanding simple sentences. Its functional advantage lies in its ability to respond swiftly in complex environments, conserve cognitive resources, and is well-suited for routine simple tasks or emergency situations. However, it is susceptible to biases, emotions, and stereotypes, potentially leading to irrational judgments, and is primarily associated with brain regions such as the limbic system (e.g., the amygdala) and the basal ganglia. In contrast, System 2 (analytical system) involves slow, controlled, conscious processing that demands active effort, relying on logical reasoning, abstract thinking, and rule application—for instance, solving complex mathematical problems or formulating long-term plans. Its strength lies in overcoming intuitive biases, enabling in-depth analysis and rational decision-making, making it suitable for complex, unfamiliar, or high-risk tasks. Nevertheless, it consumes substantial cognitive resources and operates with lower efficiency, primarily linked to brain regions such as the prefrontal cortex.

This theory holds extensive significance and applications across multiple domains. In psychology and neuroscience, it provides a framework for explaining cognitive biases, learning mechanisms, and the division of brain functions. In economics and decision science, it reveals the roots of human irrational decision-making, laying a theoretical foundation for behavioral economics. However, the theory is not without controversy: some scholars argue that the division of “two systems” is overly simplistic and have proposed a “multi-

system theory.” Its extension directions include explaining the origins of the two systems in conjunction with evolutionary psychology, or applying it to the field of education to design teaching methods that balance intuitive experience and logical training. In summary, the dual-process theory of cognitive neuroscience profoundly illuminates the complexity of human cognition, and its “intuition-analysis” binary framework not only serves as a key to understanding human thinking but also provides important insights for interdisciplinary research.

The dual-process theory directly inspires the design of the CSLF framework. In CSLF, the semantic extraction module corresponds to the “System 2” analytical process, performing deeper, more controlled semantic reasoning and abstraction. The latent feature abstraction module parallels “System 1,” rapidly and automatically capturing task-specific latent representations with minimal computational overhead. The adaptive gated fusion mechanism embodies the dynamic collaboration between the two systems, allowing the model to flexibly balance analytical and intuitive signals based on task demands and error feedback. This dual-stream architecture enables CSLF to effectively mitigate catastrophic forgetting by leveraging both deliberate, reasoning-driven generalization and fast, experience-driven adaptation, closely mirroring the complementary strengths of human cognition as described by the dual-process theory.

B Other Algorithmic Details

B.1 Task Latent Feature Abstraction Module

The Task Latent Feature Abstraction Module extracts differentiable routing features in the latent space. In our implementation, the backbone weights are frozen, the active task-specific LoRA adapter remains trainable, the selected feature layers are $\{6, 12, 18, 24\}$, and each layer output is mean-pooled before concatenation. Algorithm 1 summarizes the procedure.

B.2 Adaptive Gated Fusion Algorithm

Algorithm 2 details the adaptive gated fusion mechanism that dynamically balances the contributions from semantic and latent feature spaces.

Algorithm 2 Task Latent Feature Abstraction Module

Require: Input text x_i , selected backbone layers $\{L_1, L_2, \dots, L_k\}$, point mapper $\mathcal{M}$, task coordinate positions $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}$

Ensure: Latent feature activation vector $\mathbf{h}_{lat}$

```
1: Step 1: Hierarchical Feature Extraction
2: for  $i = 1$  to  $k$  do
3:   Extract hidden states from layer  $i$ 
4:   Mean-pool non-padding tokens:  $\mathbf{f}_i = \text{MeanPool}(L_i(x_i))$ 
5: end for
6: Step 2: Feature Fusion
7:  $\mathbf{F} = \text{Concat}(\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_k)$ 
8: Step 3: High-dimensional Mapping
9:  $\mathbf{z} = \mathcal{M}(\mathbf{F})$ 
10: Step 4: Distance Calculation
11: for  $i = 1$  to  $N$  do
12:    $d_i = \|\mathbf{z} - \mathbf{p}_i\|_2^2$ 
13: end for
14: Step 5: Activation Vector Generation
15:  $\mathbf{h}_{lat} = \text{InverseMinMaxNorm}([d_1, d_2, \dots, d_N])$ 
16: return  $\mathbf{h}_{lat}$ 
```

C Implementation Details

C.1 Network Architecture Specifications

Semantic Extraction Module.

The semantic extraction module utilizes the MiniLM-L12-v2 embedding model, which produces 384-dimensional semantic representations. It is built upon pre-trained architectures to ensure robust contextual understanding. A dynamic dictionary structure serves as the semantic pool, enabling efficient retrieval and update of semantic prototypes for each task. Additionally, a reflection threshold of $\tau = 0.7$ is employed to trigger accuracy-based revisions, ensuring that the semantic representations remain adaptive and precise throughout continual learning.

Latent Feature Module.

The latent feature module employs a multi-layer hierarchical fusion strategy, aggregating features from layers 6, 12, 18, and 24 of the backbone network. Each selected hidden state is mean-pooled over non-padding tokens, producing four vectors of size d_{hid} , which are concatenated and projected into a 256-dimensional routing space using a two-layer MLP with dimensions $4d_{hid} \rightarrow 512 \rightarrow 256$. During continual training, the backbone weights

Algorithm 3 Adaptive Gated Fusion Mechanism

Require: Semantic activation vector $\mathbf{h}_{sem}$, latent activation vector $\mathbf{h}_{lat}$, attention query $\mathbf{w}_q$, error rates e_{sem}, e_{lat} , error penalty coefficient β

Ensure: Final activation vector $\mathbf{h}_{final}$

```
1: Step 1: Initial Attention Computation
2:  $s_{sem} = \mathbf{w}_q^\top \cdot \mathbf{h}_{sem}$ 
3:  $s_{lat} = \mathbf{w}_q^\top \cdot \mathbf{h}_{lat}$ 
4:  $[\alpha_{sem}, \alpha_{lat}] = \text{softmax}([s_{sem}, s_{lat}])$ 
5: Step 2: Error Feedback Correction
6:  $\alpha'_{sem} = \alpha_{sem} \cdot (1 - \beta \cdot e_{sem})$ 
7:  $\alpha'_{lat} = \alpha_{lat} \cdot (1 - \beta \cdot e_{lat})$ 
8: Step 3: Normalization
9:  $\text{sum} = \alpha'_{sem} + \alpha'_{lat}$ 
10:  $\alpha'_{sem} = \alpha'_{sem} / \text{sum}$ 
11:  $\alpha'_{lat} = \alpha'_{lat} / \text{sum}$ 
12: Step 4: Gated Fusion
13:  $\mathbf{h}_{final} = \alpha'_{sem} \cdot \mathbf{h}_{sem} + \alpha'_{lat} \cdot \mathbf{h}_{lat}$ 
14: return  $\mathbf{h}_{final}$ 
```

are frozen and only the active LoRA adapter, the points mapper, and the learnable task points are updated. Task separation in this space is enforced via a supervised task preference loss with weight $\lambda = 0.5$ and margin $\gamma = 2.0$.

Fusion Mechanism.

The fusion mechanism employs a learnable 256-dimensional attention query vector to compute the initial fusion weights between the semantic and latent activation vectors. An error penalty coefficient of $\beta = 0.3$ is used to dynamically adjust these weights based on the observed error rates of each module. The error rates are updated every 100 training steps to ensure that the fusion process remains adaptive and responsive to the model's performance during continual learning.

C.2 Training Configuration

The hyperparameters listed in Table 4 are carefully selected to ensure stable and efficient training of the CSLF framework. The adaptive batch size and linear learning rate decay help balance convergence speed and generalization. A relatively long maximum sequence length (2048 tokens) allows the model to handle complex and lengthy inputs. LoRA-specific parameters (rank, alpha, dropout) are tuned for parameter-efficient adaptation. Warmup steps and gradient clipping further stabilize training, while a moderate weight

Parameter	Value
Batch Size	4-16 (adaptive)
Learning Rate	1e-5 (linear decay)
Max Sequence Length	2048 tokens
LoRA Rank	4
LoRA Alpha	8
LoRA Dropout	0.1
Warmup Steps	500
Total Training Steps	5000 per task
Gradient Clipping	1.0
Weight Decay	0.01

Table 4: Detailed training hyperparameters for CSLF framework

decay helps prevent overfitting. These settings collectively contribute to the robust performance of CSLF across various continual learning tasks.

D Extended Experimental Results

D.1 Per-Task Performance Analysis

The results in Table 5 clearly demonstrate the progressive improvement of CSLF over both SEQ and I-LoRA as the number of tasks increases. While all methods experience a decline in accuracy as more tasks are introduced (reflecting the challenge of catastrophic forgetting), CSLF consistently outperforms the baselines at every stage. Notably, the performance gap between CSLF and the other methods widens with each additional task, indicating that CSLF is more robust to the accumulation of tasks and better at retaining knowledge from earlier tasks. This highlights the effectiveness of the dual-stream architecture and adaptive fusion mechanism in mitigating forgetting and maintaining high accuracy throughout continual learning.

D.2 Memory Efficiency Analysis

Table 6 compares the memory efficiency of different continual learning methods. Full fine-tuning incurs the highest memory overhead, as it requires updating all model parameters for each task. Adapter and LoRA methods significantly reduce the number of additional parameters, resulting in much lower memory usage. I-LoRA and CSLF introduce slightly more parameters due to their more complex architectures, but still maintain a small fraction of the total model size compared to full fine-tuning. Notably, CSLF achieves a good balance between memory efficiency and performance, enabling scalable continual learning without ex-

cessive resource consumption. This makes CSLF suitable for real-world applications where memory and computational resources are limited.

D.3 Convergence Analysis

The convergence behavior of CSLF shows stable learning curves across different tasks. The semantic module typically converges within 1000 steps, while the latent feature module requires 2000-3000 steps for optimal performance. The adaptive fusion mechanism continuously adjusts throughout training, with error rates stabilizing after approximately 1500 steps.

D.4 Inference Overhead Analysis

To quantify deployment cost, we compare CSLF with a strong parameter-separation baseline (I-LoRA) on test-set inference efficiency. We report per-sample latency (ms/sample, lower is better) and throughput (samples/s, higher is better). All results are measured on a single NVIDIA A100 (80GB), batch size 1 for latency and batch size 32 for throughput.

As shown in Table 7, CSLF achieves better continual-learning accuracy at the cost of higher inference latency and lower throughput than I-LoRA. This overhead mainly comes from dual-stream activation computation and gated fusion. Therefore, inference efficiency is a current limitation of CSLF in latency-sensitive scenarios.

E Error Analysis and Case Studies

E.1 Common Error Patterns

Through detailed analysis of misclassified samples, we identify three main error categories:

Task	SEQ	I-LoRA	CSLF	Improvement
Task 1	89.2	91.3	94.1	+2.8
Task 2	67.4	78.9	85.2	+6.3
Task 3	45.1	69.7	79.4	+9.7
Task 4	32.8	62.1	75.8	+13.7
Task 5	21.3	55.4	72.3	+16.9
Task 6	15.7	48.6	68.9	+20.3
Task 7	12.1	43.2	65.7	+22.5

Table 5: Per-task accuracy on Banking77 dataset showing progressive improvement

Method	Additional Parameters	Memory Overhead
Full Fine-tuning	8B (100%)	High
Adapter	3.2M (0.04%)	Low
LoRA	2.1M (0.026%)	Low
I-LoRA	4.8M (0.06%)	Medium
CSLF	6.3M (0.079%)	Medium

Table 6: Memory efficiency comparison across different methods

Semantic Ambiguity: Cases where multiple tasks share similar semantic concepts but require different classifications. Example: “transfer money” could belong to both “card payment” and “transfer into account” tasks.

Fusion Conflicts: Situations where semantic and latent modules disagree significantly, and the fusion mechanism fails to select the optimal combination.

E.2 Case Study: Banking77 Error Analysis

In the Banking77 dataset, we observe that 78% of errors are concentrated in semantically related categories, such as “get physical card” versus “order physical card”, “compromised card” versus “lost or stolen card”, and “top up failed” versus “top up reverted”. These confusions highlight the challenge of distinguishing between tasks with highly similar semantic content.

CSLF’s dual-stream approach reduces these errors by 34% compared to single-stream methods, demonstrating the effectiveness of combining semantic and latent information.

F Hyperparameter Sensitivity Analysis

F.1 Error Penalty Coefficient β

Table 8 shows the performance sensitivity of CSLF to the error penalty coefficient β . A value of $\beta = 0.3$ consistently yields the best results across all datasets, balancing the influence of semantic

and latent features effectively. Lower values lead to underutilization of latent features, while higher values cause excessive reliance on semantic features, resulting in suboptimal performance.

F.2 Task Preference Loss Weight λ

The task preference loss weight λ significantly affects the separation quality in the latent space. Our experiments show that $\lambda = 0.5$ provides the optimal balance between attraction and repulsion terms across all datasets.

G Future Research Directions

Based on our work, several promising research directions emerge:

Multi-modal Integration: Extending CSLF to handle multi-modal inputs (text, images, audio) within the same framework.

Dynamic Architecture Growth: Investigating methods to automatically expand the network architecture as new tasks arrive.

Meta-learning Integration: Combining CSLF with meta-learning approaches for faster adaptation to new tasks.

Theoretical Analysis: Developing theoretical guarantees for catastrophic forgetting bounds in dual-stream architectures.

Method	Latency (ms/sample) ↓	Throughput (samples/s) ↑
I-LoRA	23.4	136.8
CSLF (ours)	31.7	101.5

Table 7: Inference efficiency comparison on the test set (Llama-3-8B backbone).

β	Banking77	FewRel	CLINC150
0.1	72.4	67.2	74.1
0.2	74.1	68.9	76.3
0.3	75.1	69.4	78.8
0.4	74.6	68.7	77.9
0.5	73.2	67.5	76.4

Table 8: Performance sensitivity to error penalty coefficient β

H. Reproducibility Details

H.1 Code and Data Availability

All experimental code, trained models, and detailed experimental logs will be made available upon publication. The implementation is based on PyTorch and Transformers libraries.

H.2 Hardware Requirements

To facilitate reproducibility, we report the hardware configuration used in our experiments and provide practical minimum requirements.

Minimum.

- **GPU:** NVIDIA GPU with at least 16 GB memory
- **CPU/RAM:** 32 GB system memory
- **Storage:** 100 GB for datasets and checkpoints

Recommended (as used in this work).

- **GPU:** NVIDIA A100 (80 GB)
- **CPU/RAM:** 128 GB system memory
- **Storage:** 500 GB SSD

H.3 Semantic Routing Prompt and Update Rules

The semantic stream uses the backbone LLM as a prompted task router. For each sample x_i , we first rank all observed tasks by MiniLM cosine similarity and keep the top- m candidates ($m = 5$ in all experiments). The routing prompt has the following form:

Task candidates:

Task 1: [task description]

Exemplars: [short exemplar 1]; [short exemplar 2]

...

Task m : [task description]

Exemplars: [short exemplar 1]; [short exemplar 2]

Input: [raw text or task-formatted input]

Question: Which task ID best matches the input? Answer with one ID only.

The backbone generates one task ID token/string with greedy decoding, and we convert the normalized log-probabilities of all candidate IDs into the semantic activation vector $\mathbf{h}_{sem}$. This forward pass is used only for routing and semantic-pool revision; it is not differentiated and does not update the backbone parameters.

When the batch routing accuracy falls below τ , we trigger reflection-revision. The procedure is deterministic after error collection:

1. Collect all misrouted samples and group them by (gold task, predicted task) pair.
2. For each pair, retain the top- b samples with the highest routing confidence ($b = 3$ in our experiments).
3. Append those samples to the exemplar buffers of the corresponding gold tasks.
4. Rebuild each task description with the prompt below and replace the old description in $\mathcal{S}$.

You are revising a task description for routing.
 Task label set: [labels]
 Correct exemplars: [examples]
 Most confused neighboring task: [task text]
 Write a short description containing: the common topic, the boundary to the confused task, and 3–5 lexical cues.

This update changes only the textual task description and exemplar buffer. It does not alter any model weights.

H.4 Differentiability and Optimization Scope

CSLF uses two training paths with different optimization behavior:

- **Semantic stream:** MiniLM embeddings, nearest-neighbor retrieval, prompt construction, and prompted backbone task routing are all treated as non-differentiable operations. They are executed in forward mode only.
- **Latent stream:** the active task-specific LoRA adapter, the points mapper $\mathcal{M}$, the task points $\{\mathbf{p}_T\}$, and the fusion parameters are trained with backpropagation.
- **Frozen parameters:** all original backbone parameters remain frozen throughout continual training.

The task preference loss is therefore a supervised routing objective, because each training sample is associated with its current task ID. At inference time, task IDs are unknown and routing is performed by the fused activation vector.

H.5 Tensor Shapes and Initialization

For the semantic stream, the MiniLM embedding dimension is 384 and the semantic activation vector has size $|\mathcal{T}_{\leq t}|$, where $|\mathcal{T}_{\leq t}|$ is the number of tasks observed so far. For the latent stream, if the backbone hidden size is d_{hid} , each pooled layer feature has size d_{hid} , the concatenated feature has size $4d_{\text{hid}}$, and the mapper output has size 256.

Each new task point $\mathbf{p}_T \in \mathbb{R}^{256}$ is initialized from $\mathcal{N}(0, 0.02^2)$ when task T first appears. The point remains learnable afterward so that the routing geometry can be refined as more tasks are observed.